

• ADFORGE AI — PROJECT DOCUMENTATION

• Code-Layout, Readability and Reusability

- **Date:** 7th July 2026
- **Development Profile:** Sunehri Sharma
- **Maximum Marks:** 5 Marks

Proposed Code Quality & Structure Matrix

S.No	Quality Principle	Implementation Details / Description	Specific Project Examples (Files/Folders)
1	Folder/Directory Structure	Lightweight, flat-structured architecture optimized for micro-web applications. Separates core backend server routing, secure token storage configurations, and public client interface elements.	• Root directory containing app.py (Flask core core logic) and .env (private configuration keys). • /static folder managing frontend web files (index.html, style.css, script.js).
2	Naming Conventions	Enforces a clear split: standard lowercase with underscores (snake_case) for Python functions/endpoints, and descriptive multi-word camelCase for frontend JavaScript variables and element selectors.	• Python function: generate_ad_copy() and route handling path /api/refine in app.py. • JavaScript array: selectedPlatforms and tracking trigger fetchCopyVariations() in script.js.
3	Code Readability & Comments	Strategic inline comments outlining strict JSON template formatting rules, prompt injection layers, and complex DOM string	• Multi-line Python docstrings inside app.py outlining the exact LLaMA 3.3 model prompt parameters and character

S.No	Quality Principle	Implementation Details / Description	Specific Project Examples (Files/Folders)
		assembly behaviors to ease future evaluation and maintenance.	boundary conditions.
4	Modularity & Reusability	Strict adherence to the DRY (Don't Repeat Yourself) software principle. Isolates dynamic UI card compilation, copy operations, and fetch requests into specialized, reusable structures.	Reusable JavaScript layout generator function <code>renderOutputCard(platform, text)</code> invoked fluidly to draw Google, Facebook, and Instagram UI panels.
5	Formatting & Linting Tools	Uses native automated style engines and visual code formatting extension rules to maintain uniform indentation and spacing parameters across the workspace files.	Python Black code formatter and built-in JavaScript rules configured locally inside the <code>.vscode/settings.json</code> workspace settings file.
6	Error Handling Patterns	Comprehensive fallback checks isolating dynamic operations from total platform failure. Gracefully intercepts connection errors or model processing glitches without breaking the user view.	try-except containment blocks wrapped around Groq API calls in <code>app.py</code> that catch network timeouts and send clean error codes to frontend <code>catch()</code> handlers.